



Potential of Serious Games to Support Hyperfocus or Flow Experience in Programming Exercises

Aditya Fadhillah (hfg708)

Qianqian Fang (tgq379)

Professor: Pernille Bjørn

Project outside the course scope (PUK)

Department of Computer Science, July 21, 2026

Acknowledgments

We would like to express our sincere gratitude to all those who contributed to the success of this project.

First, we extend our thanks to Philippe Bonnet, course coordinator of Programming and Problem-Solving (PoP), and the teaching assistant from PoP for their valuable insights during the preliminary knowledge gathering phase and for providing access to students for our initial interviews.

We are especially grateful to Daniel Spikol, course coordinator of Python Programming for Data Science (PPDS), for welcoming our research into his course, facilitating access to students, and providing valuable feedback throughout the testing period. We also sincerely thank all the students from PPDS who participated in this study by using the PyMon application and sharing their experiences through interviews. Their engagement and honest feedback were invaluable to this research.

We would like to express our deepest appreciation to our supervisor, Pernille Bjørn for guiding us throughout the entire process of this project. Her constructive feedback, insightful suggestions, and overall support were key in shaping this project.

Figures, Appendices, and Transcripts

Please note that the resolution of some figures and appendices in this project may not be optimal for viewing. For access to the original figures, appendices, and the full transcripts, please refer to the following link: [Google Drive](#).

(<https://drive.google.com/drive/folders/148YSWjuZcwPrxxHZ1Mbl4SzYVGZMLmVn?usp=sharing>)

For accessing to PyMon, please refer to the following link: [PyMon](#) and the code for it can be found: [GitHub for PyMon](#).

(<https://aditfadhillah.github.io/PyMon/>)

(<https://github.com/AditFadhillah/PyMon/>)

Abstract

Neurodivergent students face significant challenges in standardized programming education, often leading to higher dropout rates. This study explores the potential of serious-gaming approaches to support STEM students in experiencing hyperfocus or flow while working on programming assignments.

We developed PyMon, a web-based serious game prototype integrating Python programming exercises with Pokemon-inspired gameplay mechanics. Through a one-week pilot study with 41 registered participants (22 active participants), we conducted mixed-method research combining usage data analysis with interviews of four students and one course instructor.

Our findings identify three mechanisms through which game-based design may influence engagement: (1) facilitating task initiation by reducing practical and psychological barriers, (2) minimizing task shifting through context-preserving design, and (3) supporting diverse motivation profiles.

While focused on neurodivergent needs, this study demonstrates that designing for hyperfocus and flow has the potential to elevate learner engagement in programming exercises. The understanding of mechanisms that support these deep focus states provides a foundation for universal design, ultimately creating a more inclusive and effective programming environment for a diverse range of students.

Contents

1	Introduction	1
2	Theory	2
2.1	Serious Games and Its Usage	2
2.2	Neurodiversity and ADHD	2
2.3	Hyperfocus and Flow	3
2.4	Game-Design Patterns	3
3	Method	4
3.1	Empirical Case - Neurodiversity in SCIENCE of University of Copenhagen	5
3.2	Data Collection & Research Activities	5
3.3	Data Analysis	6
3.3.1	Quantitative Usage Analysis	6
3.3.2	Qualitative Thematic Analysis	7
4	Design	7
4.1	Overview	7
4.2	Core Gameplay Loop and Learning Mechanics	7
4.3	Design Principles Supporting Hyperfocus and Flow Experience	7
4.4	Pretesting and Refinement	8
4.5	Technical Implementation	9
5	Results	10
5.1	Lower Start-Up Friction Facilitates Entry into Deep Focus Experience	10
5.2	Context-Preserving Breaks Support Sustained Engagement	12
5.3	Game Design Elements Dis- or Encourage Engagement but Show Individual Variation	14
5.4	Course Instructor Perspective	17
6	Discussion	18
6.1	Facilitating Task Initiation Through Reduced Barriers	18
6.1.1	Practical and Cognitive Start-Up Friction	18
6.1.2	Programming as Play	19
6.2	Minimizing Task Shifting Through Context-Preserving Design	19
6.3	Supporting Diverse Motivation Profiles	19
6.3.1	Individual Variation in Game Element Effectiveness	19
6.3.2	Balancing Competitive and Mastery	20
6.4	Educational Trade-offs: Task Chunking vs. Problem-Solving Development	20
6.5	Hyperfocus and Flow: Evidence and Limitations	20
6.6	Institutional and Adoption Challenges	21
6.7	Implications for Design and Practice	21
6.8	Universal Design and Neurodiversity	21
6.9	Future Directions	21
7	Conclusion	22
8	Appendix	24
A	Interview Guide	24
A.1	Research Question	24
A.2	Interview Questions	24
A.3	Wrap-up Question	25
A.4	Theoretical Framework Indicators	25
A.4.1	Hyperfocus (Adult Hyperfocus Questionnaire - AHQ)	25
A.4.2	Flow (Dispositional Flow Scale-2 - DFS-2)	25
B	Prototype Interface	26

1 Introduction

Programming education faces significant challenges in supporting diverse student populations. Students in computer science courses represent a neurodiverse group that includes both neurotypical learners and neurodivergent individuals with conditions such as ADHD, autism spectrum disorder, and dyslexia [5]. Neurodivergent students are overrepresented in STEM fields [24, 16] yet face higher dropout rates due to standardized teaching approaches that often mismatch their cognitive profiles [5]. At the University of Copenhagen, the number of students receiving special-pedagogical support increased by 160.25% between 2018 and 2022, with similar growth at the SCIENCE faculty [4]. These students often experience isolation and burnout when course structures do not accommodate differences in executive functioning, attention, or energy levels [5]. Traditional programming courses create barriers through multiple preparatory steps, rigid pacing, heavy workloads, and limited multimodal materials, creating unnecessary cognitive overload that makes it harder for neurodivergent learners to sustain focus [5].

However, research also documents that neurodivergent individuals, particularly those with ADHD, can experience hyperfocus, which is intense and sustained attention that enables deep, productive engagement when activities feel rewarding and interesting [12, 21, 10]. Serious games present a promising opportunity to leverage this potential. Serious games combine interactive game environments with structured educational goals, using game-based techniques such as immediate feedback, goal achievement, and competition to sustain engagement and attention [9, 18]. Research demonstrates that serious games can enhance executive functions including working memory, inhibitory control, and cognitive flexibility for neurodivergent populations [18, 1]. The game-based approach may create conditions conducive to flow states, which are characterized by clear goals, immediate feedback, and balanced challenge, or support the emergence of hyperfocus through engaging, rewarding task structures [2, 11]. However, while serious games have been explored for teaching specific programming topics, their potential to support sustained focus experiences during programming practice remains underexplored.

This leads to our research question:

What are the potentials of using serious-gaming approaches to support STEM students in experiencing hyperfocus or flow while working on programming assignments?

To explore this question, we first unpack theories including serious games, neurodiversity and ADHD, hyperfocus and flow, and game design patterns. Based on these theoretical foundations, we conducted mixed-method research, including informal and semi-structured interviews with a course instructor, a teaching assistant, and students to collect insights for the serious game prototype. Furthermore, we developed PyMon¹, a web-based serious game prototype that integrates Python programming education with Pokemon-inspired gameplay mechanics (Fig. 1). We conducted a one-week pilot study with 41 students from an introductory Python programming course at the University of Copenhagen, collecting usage data and conducting semi-structured interviews with four students and the course instructor.

We analyzed usage patterns from PyMon combined with user experiences gathered through semi-structured interviews with participants. Through this analysis, we identified that the gap between engagement initiation and engagement enactment influences the occurrence and sustainment of focus. Specifically, we found three key factors that affect this gap for students while doing programming exercises: **(1)** physical and psychological barriers that hinder task initiation; **(2)** different types of breaks that influence the continuity of engagement; and **(3)** game design elements that support the continuity of engagement, though in highly individualized ways.

We discuss these findings in relation to theoretical frameworks, and suggest implications for designing game-based programming education tools that support sustained engagement in programming exercises. These findings contribute to our understanding of how universal design principles informed by neurodiversity can create more inclusive programming education that benefits all students [5].

The rest of the paper is structured as follows: Section 2 presents theoretical foundations in serious games, neurodiversity and ADHD, hyperfocus and flow, and game design patterns. Section 3 describes our research methods. Section 4 introduces our prototype PyMon and explains its design choices for supporting hyperfocus and flow experience. Section 5 presents results identifying mechanisms that influence engagement. Section 6 discusses how findings relate to theoretical frameworks, addresses educational trade-offs, examines institutional barriers, and proposes design considerations. Section 7 concludes by reflecting on the potential of serious gaming approaches to create more inclusive programming education environments.

¹<https://aditfadhillah.github.io/PyMon/>

2 Theory

This section presents the theoretical concepts that underpin our research question on what design principles might support hyperfocus states among neurodivergent Computer Science students learning programming.

The discussion connects theories from serious games, neurodiversity, hyperfocus, and game-design patterns, by examining how these perspectives might relate to sustained engagement and the support of hyperfocused learning states.

2.1 Serious Games and Its Usage

Serious games (SGs) are games designed for a primary purpose beyond entertainment, most commonly, they are designed for learning, training or behaviour development. SGs make use of game-based techniques such as immediate feedback, competition, goal achievement and collaboration to sustain user engagement and attention. Research on instructional games shows that these features can trigger iterative cycles of user judgment, behaviour, and feedback which leads to sustained engagement in users and their persistence over time for practicing. [9]

By combining interactive game environments with structured educational goals, SGs allow players to learn and practice specific skills through different scenarios that are purposely designed to support the learning process. This creates a hybrid experience that supports both enjoyment and skill acquisition. [18] Sustained engagement is a critical mechanism distinguishing SGs from more traditional instructional approaches, as engagement in game-based activities has been shown to increase attention to learning content and contribute to the achievement of learning outcomes by persistence practicing. [9]

Research has repeatedly shown that serious games increase learner motivation, engagement and provides tailored, personalized learning experiences. [1] Engagement in this context is not limited to enjoyment, but includes deep task involvement, focused attention and persistence, which are closely linked to effective learning. [9]

SGs have also demonstrated effectiveness in supporting cognitive development. According to research (Rodríguez Timaná et al. [18] (2004)), SG interventions have been shown to enhance executive functions including working memory, inhibitory control and cognitive flexibility for individuals with neurodiverse conditions.

These benefits are especially evident among neurodiverse populations. Serious games interventions have been shown to enhance cognitive abilities such as attention, working memory, planning skills and emotional regulation in children with ADHD and other neurodiverse populations. Hence, there is a growing trend of use of SG as a powerful tool for educational and therapeutic applications within neurodiversity landscape. [18, 1]

Because of this established connection, it is necessary to examine the neurodiversity population more closely to understand their specific attentional and cognitive challenges faced by these learners. This provides the foundation for understanding why SG-informed design approaches may be particularly relevant for supporting neurodiverse learners, including those studying computer science.

2.2 Neurodiversity and ADHD

Neurodiversity describes the variability within human neurological development. This emphasizes that individuals differ in how they learn, think, and process information. Rather than viewing these differences as deficits, the neurodiversity perspective recognizes conditions such as ADHD, autism spectrum disorder, dyslexia, and Down syndrome as distinct neurological profiles with unique strengths and challenges. Additionally, research emphasizes that even though the neurodiverse population share some common traits, their strengths, challenges and responses to interventions are highly individual, which means that they would need more tailored assistance. [18]

This recognition of neurological variability is particularly relevant for understanding how learners experience cognitively demanding domains such as computer science and programming.

Neurodivergent individuals often exhibit challenges in various executive functions. Executive functions describe the cognitive processes that are used for regulating and controlling adaptive and goal-directed behavior, this including, working memory, ability to inhibit behaviors, task switching, planning, emotional regulation, impulse control and motivation. Executive functions play a crucial part in individuals daily functioning, social interaction and academic success. Hence, the neurodiverse individuals often show weakness in organization, monitoring progress and work quality. [18]

Consider Attention-Deficit/Hyperactivity Disorder (ADHD), one of the most widely studied neurodivergent conditions. ADHD is a neurodevelopmental condition characterized by inattentiveness,

hyperactivity, and impulsiveness. It has been observed that children with ADHD have a shortened span of working memory, which is the ability to store, process, and generate responses to information. They are more likely to struggle to finish activities as they get bored quickly. Hence, it is common for students with ADHD to underperform in academia and experience other educational problems. As one of the consequences, students with ADHD often require cognitive behavior therapies to assist them in better performance. [1, 14]

Research across STEM further illustrates that neurodivergent learners are not a single profile, yet patterns do emerge. Autistic students are over-represented in STEM fields while enrolling in college at lower rates overall, especially within the CS fields. [24, 16] CS students often struggle not because of a lack of ability, because their cognitive profiles misaligned with how standardized CS courses are structured. Course staff are frequently unable or insufficiently supported to address these needs, resulting in environments that do not anticipate differences in executive functioning, attention, or energy levels. The rigid pacing, heavy workloads, and lack of multimodal materials create unnecessary cognitive overload, making it harder for neurodivergent learners to sustain focus and making them more vulnerable to cognitive fatigue. [5]

These findings suggest strong interest and aptitude to explore different learning design approaches to coexist with barriers that the neurodivergent learners might face to minimize the friction and create better learning.

These structural and situational barriers also shape how attention fluctuates during study. In particular, they may explain when and how hyperfocus arises, which can enable deep, productive engagement, and at other times making it harder to initiate or switch tasks. This dynamic will be further explored in the next subsection.

2.3 Hyperfocus and Flow

Hyperfocus describes episodes of intense and sustained attention in which attention becomes narrowly locked onto a task. People often report reduced awareness of their surroundings, a distorted sense of time, and difficulty disengaging or switching tasks once the episode is underway [12, 21]. Hyperfocus has frequently been discussed in relation to ADHD [19, 15], but empirical work also indicates that it is not exclusive to ADHD and can appear in the general population [10]. Importantly for education, hyperfocus does not necessarily occur in academic contexts as often as in personally rewarding activities, and it may be harder to initi-

ate or sustain in learning situations that are experienced as stressful, unclear, or overly demanding [10].

A closely related concept is flow, which is often described as being "in the zone", indicated by deep concentration on an activity accompanied by smooth task involvement, reduced distraction, and altered time perception. Because both hyperfocus and flow can involve intense attention and time distortion, the two are sometimes treated as overlapping experiences [2, 11]. However, recent research emphasizes that they can also be distinguished as different experiences [11]. A key theoretical difference is that flow is typically framed as an optimal and controlled engagement state: it is supported when goals are clear, feedback is immediate, and challenges are perceived as manageable relative to one's skills [2]. In contrast, hyperfocus is often characterized by a stronger sense of attentional locking and greater difficulty disengaging, which can be productive for performance in the moment but may be less flexible or harder to regulate [12, 2].

These distinctions matter for designing learning tools. If the aim is to support productive, sustained engagement during programming practice, flow theory points toward design features that make task goals explicit, provide rapid feedback on progress, and calibrate challenge to skill level [2]. Hyperfocus research, meanwhile, highlights both the potential upside of prolonged concentration [12, 11]. These concepts also informed how we assessed participants' reported experiences in the qualitative analysis, where we looked for interview evidence consistent with established descriptions of hyperfocus and flow. [11] Together, these perspectives provide a lens for reasoning about how serious game design elements might increase the likelihood that students experience deep engagement while working on programming assignments. Furthermore, these concepts provide insights into which game design patterns should be selected and integrated in SG to effectively facilitate or scaffold hyperfocus and flow experience for students when working with SG.

2.4 Game-Design Patterns

In order to design for hyperfocus, it is essential to examine what kinds of design pattern have been used in current practices to influence players' behavior both in games and digital products.

Different design strategies can be used in games and digital products to influence user behavior and engagement or even manipulate users. These strategies range from **dark patterns** and **addictive patterns** to **positive behavioral design**

patterns.

Dark design can be described through four main categories of dark patterns; temporal, monetary, social and psychological, adapted from the dark-pattern literature in games [25]. Addictive patterns are considered as a subset of dark patterns, with the same purpose of maximizing user engagement but more oriented toward making users spend more time using the product or have a greater degree of commitment than what is expected, convenient or health for them. [3]

It is important to draw a line between dark patterns and addictive patterns regarding decision autonomy. Dark patterns distort user decisions by exploiting cognitive biases. In contrast, addictive patterns are specifically aimed at producing compulsive engagement and user dependence. [3]

Viewed this way, addictive patterns can be repurposed for positive outcomes that benefit the player, such as supporting focus and persistence for learning. For the temporal engagement, forced action pattern can be applied in terms of Gamification. In an educational setting, timer, soft streaks, and productive feedback loops, leaderboard system, progress bars and mastery badges can help support motivation. [13].

These game-like mechanism make uses the behavioral change techniques. These behavioral change techniques includes:

Reinforcement and punishment includes the process of strengthening or weakening behaviors through rewards and feedback;

Nudging and facilitating, make desirable actions easier or more likely to occur.

Social techniques & modeling, where the player's action can be influenced by observing others' action or social norms.

Complex feedback systems, the system should provide continuous feedback about progress and performance, such as graded task, adjust the level's difficulty, and contingent rewards.

Guided practice, makes it available for users to repeat behaviors at various times and provides them with feedback. [23]

Scaffolded learning, supports sustained engagement in programming practice by reducing the effort required to get started. This can be achieved through Worked-out examples and scaffolding. [20]

Viewed as a whole, these contrasting pattern groups illustrate that the same behavioral mechanisms used to manipulate attention or induce compulsive engagement can also be adapted to pro-

mote constructive and intentional learning outcomes. With this in mind, these techniques serve as the conceptual foundation for identifying which design elements may help neurodivergent learners, particularly those with ADHD, enter and sustain productive hyperfocused learning states by designing the learning in serious games to be more engaging.

In this study, we bridge these behavioral theories with our investigation of neurodivergent learning by adopting the concepts of Reinforcement, Nudging, and Social technique from positive behavioral design patterns and concrete implementation elements such as timer, leaderboard system, and badges from Gamification as a conceptual framework. These concepts provide the potential to influence students' behavior while starting programming exercises, which should encourage them to spend more time and be more engaged. This framework provides the analytical dimensions required to study how such design patterns influence student behavior and experience of hyperfocus and flow experience during programming tasks and whether they foster the necessary conditions for sustained engagement.

3 Method

To explore the potential of serious-gaming approaches to support experiences of hyperfocus or flow while working on programming tasks for STEM students, this project employs an exploratory mixed-methods research design[22] with data triangulation [17].

This project is divided into three phases: **1) Understanding the Problem:** pre-interviews with introductory programming courses teachers, one teaching assistant, and students to gather knowledge about current practices and challenges in programming learning, which informs the prototype design; **2) Designing a Solution:** Prototype design, implementation and testing based on insights collected from pre-interviews, relevant theoretical frameworks, and neurodivergent student assessments; and **3) Evaluating the Solution:** in-class observation and post-interviews with introductory programming courses teachers and students following the use of the prototype to explore users' experiences with the prototype to generate insights for data analysis. The following sections first introduce the empirical context of this project which is about the high ratio of neurodivergent students in STEM disciplines within University of Copenhagen. This empirical context provides the motivation for the study. Next, we describe data collection, research activities and data analysis procedures conducted in this project.

3.1 Empirical Case - Neurodiversity in SCIENCE of University of Copenhagen

Neurodivergent students exist among neurotypical students and have a surprisingly high representation rate in both STEM and computer science fields. Statistics from NeuroGen Computing show that the number of diagnosed students receiving 'special-pedagogical support' (SPS) has increased rapidly at the University of Copenhagen — by 160.25% between 2018 and 2022 (from 863 to 2,246 students), and at the SCIENCE faculty by 157.30% (from 274 to 863 students).[4]

The most common reasons for students receiving SPS support are dyslexia and "mental impairment," which is the term used in the SPS classification system covering neurodiversity. Due to conventional teaching approaches which often mismatch the needs of neurodivergent profiles, these students are often exposed to experiences of isolation, stigma, and burnout, leading to increased risks of leaving computer science programs. [5, 4]

With these observed challenges faced by the neurodivergent students in mind, this project aimed to collaborate with introductory programming courses for further domain knowledge gathering.

First, the project group reached out to the introductory programming course NDAB15009U Programming and Problem-Solving (PoP) at Department of Computer Science (DIKU). The PoP course is the mandatory introductory programming course for bachelor students who are majoring in Computer Science (207), Computer Science and Economy (102), Machine Learning and Data Science (100), and Cognition and Data Science (45). The approximate number of students for this course is around 400 people given by the number of year 2024. [8]

During our preliminary knowledge gathering and research design consultation with the course coordinator, TA, and a student of PoP, we acknowledged that the PoP students have different backgrounds and levels of programming experience. Many of the students who have already very high levels of programming experience would like to obtain a formal education; therefore, their motivation for learning programming is already high. Thus, this led the project group to reach out to another introductory programming course — NBIB20000U Python Programming for Data Science (PPDS).

One of the aims of collaborating with the SCIENCE introductory course NBIB20000U Python Programming for Data Science (PPDS) was to have a larger target group of students without

any coding background or with very limited coding experience. Second was to aim for a larger distribution of STEM majors compared to being limited within Computer Science related majors. Even though the PPDS had a smaller number of attendees (120), it had a broader range of students from different majors since it is a restrictive elective course for molecular biomedicine bachelor students and other SCIENCE related courses, but it was also taken by students from mathematics, Computer Science, Health Informatics, and other fields as electives. Therefore, this target group has the potential to provide more varied levels of motivation for learning programming, which is desirable for conducting our research to explore the potential of serious gaming in supporting them in experiencing hyperfocus or flow while working on programming tasks.

3.2 Data Collection & Research Activities

This study draws on two primary types of data: **1) data for prototype design** and **2) data for evaluating the prototype**. In order to collect the *data for prototype design*, we conducted different activities for the project phases of *Understanding the Problem & Designing a Solution*.

During the *Understanding the Problem* phase, we carried out an informal interview [7] with introductory programming courses (PoP & PPDS) teachers, semi-structured interviews with one teaching assistant (PoP), and students (PoP) to gather insight about current practices, challenges, and their experience in programming learning, which provided the insight for prototype baseline design. Furthermore, we iteratively refined the prototype during the *Designing a Solution* phase with neurodivergent students (who had more programming experience) providing assessments and feedback on earlier versions of the prototype in order to minimize bugs and ensure that the desired functionality of the prototype was fulfilled and ready to be presented to the target group.

To collect *data for evaluating the prototype*, we conducted data collection during both the *Designing a Solution* and *Evaluating the Solution* phases. Note that some overlap occurred with *data for prototype design* collection during *Designing a Solution*, as we iteratively refined the prototype until it was ready for the target group.

During the *Evaluating the Solution* phase, we conducted in-class observations over a one-week period, during which the research group was present in PPDS exercise classes to introduce PyMon and observe student interactions with the application. Students could then use PyMon asynchronously throughout the week. The application automat-

Date(s)	Participants	Activity	Phase	Data Type
12.08.2025	1 PoP Course Coordinator	Informal interview	Understanding the Problem	Prototype Design
02.10.2025	1 PoP Student	Semi-structured interview	Understanding the Problem	Prototype Design
08.10.2025	1 PoP TA	Semi-structured interview (online)	Understanding the Problem	Prototype Design
09.10.2025	1 Python Course Coordinator	Informal interview	Understanding the Problem	Prototype Design
11.11.2025	1 Python Course Coordinator	Informal interview (online)	Understanding the Problem	Prototype Design
24.11.2025	1 Neurodivergent Comp-Sci MSc Student	Written feedback	Understanding the Problem & Designing a Solution	Prototype Design & Evaluation
26.11.2025	1 Neurodivergent Comp-Sci MSc Student	Written feedback	Understanding the Problem & Designing a Solution	Prototype Design & Evaluation
28.11.2025	3 CompSci MSc Students	Written feedback	Understanding the Problem & Designing a Solution	Prototype Design & Evaluation
01–07.12.2025	41 Python Course Students	PyMon application usage observation	Evaluating the Solution	Prototype Evaluation
08.12.2025	1 Python Course Student (1st year Bioinformatics)	Semi-structured interviews	Evaluating the Solution	Prototype Evaluation
08.12.2025	1 Python Course Student (3rd year Mathematics)	Semi-structured interviews	Evaluating the Solution	Prototype Evaluation
10.12.2025	1 Python Course Student (3rd year Molecular Biomedicine)	Semi-structured interviews	Evaluating the Solution	Prototype Evaluation
10.12.2025	1 Python Course Student (3rd year Molecular Biomedicine)	Semi-structured interviews	Evaluating the Solution	Prototype Evaluation
11.12.2025	1 Python Course Coordinator	Semi-structured interview	Evaluating the Solution	Prototype Evaluation

Table 1: Overview of research activities and data collection timeline.

ically collected quantitative usage data during this period, including session timestamps, session durations, task completion, Pokemon captures, points earned, and team affiliations. This dataset provides measurement of engagement and activity to uncover potential trends in user activities that correlate with the hyperfocus or flow experiences during solving the programming tasks. Furthermore, we conducted semi-structured interviews with introductory programming courses teachers and students following the use of the prototype to explore users’ experiences with the prototype to generate insights for data analysis related to the experience of hyperfocus and flow while using the prototype.

The research activities for this project are listed in Table 1 sorted by date and grouped by Participants, Activities, Phase and Data types.

3.3 Data Analysis

The study employs a mixed-methods analytical approach [22], combining quantitative user usage pattern analysis of prototype with qualitative thematic analysis [6] of post-experience interview transcripts.

3.3.1 Quantitative Usage Analysis

Application log data from registered players was analyzed to identify temporal activity patterns, session duration, and indicators of sustained engagement. The analysis covered session frequency and duration, task completion rates, team affiliation and leaderboard position, and progression signals such as points accumulation and Pokémon capture rates. These metrics provide an objective baseline for when and how long students engaged with PyMon.

Session Duration Map and Engagement Depth Map To examine engagement patterns more closely, we analyzed activity and capture data from the four interview participants (L, C, M, E). Figure 3 displays both the active session durations and Pokemon capture timestamps for these participants throughout the week.

Analysis of these patterns reveals distinct engagement strategies among the four participants. The data show two primary engagement patterns: *distributed multi-day engagement* and *concentrated single-day engagement*.

The contrast between session start patterns (Fig-

ure 2) and capture patterns (Figure 3) suggests that session activity alone may not fully reflect active engagement. Some sessions show minimal or no capture activity, which could indicate several possibilities: participants may have left the web application open when closing their laptop (causing the app to register continued session time when the device was reopened), or they may have spent time exploring the interface, reading instructions, or encountering technical difficulties. These observations suggest that actual task completion (captures) may provide a more reliable indicator of active engagement than session activity alone.

3.3.2 Qualitative Thematic Analysis

Interview transcripts were analyzed using thematic analysis [6] following an iterative coding process. Transcripts were initially coded inductively to identify emergent patterns, then refined through comparison and used concepts from prior work distinguishing hyperfocus and flow experiences to guide interpretation [11].

To connect interview data to established constructs, excerpts were tagged using operational indicators aligned with AHQ- and DFS-2-style constructs [11]. Hyperfocus indicators captured; (H1) intense task locking, (H2) reduced awareness of surroundings, (H3) time transformation, and (H4) difficulty disengaging. Flow preconditions captured; (F1) clear goals, (F2) performance feedback, (F3) challenge-skill balance, (F4) sense of control, and (F5) intrinsic reward. These tags were used to organize and interpret qualitative evidence of participants' reported experiences during programming practice.

4 Design

This section presents the web-based serious gaming prototype and explains the design principles for supporting hyperfocus and game design patterns that promote sustained engagement.

4.1 Overview

The prototype, PyMon², is a web-based serious game that integrates Python programming exercises with Pokemon-inspired gameplay mechanics. PyMon features a two panel interface where gameplay and code development occur simultaneously, creating an integrated learning environment designed to support hyperfocus or flow experiences during programming practice.

As shown in Figure 1, the interface is divided into two main sections: the game world (left panel)

²<https://aditfadhilah.github.io/PyMon/>

and the Python code editor with syntax highlighting (right panel). Students explore a 2D overworld map using keyboard controls (ARROW KEYS or WASD), encountering wild Pokemon in four distinct regions: Forest, Beach, Volcano, and Swamp. Each region contains unique Pokemon types and presents programming challenges aligned with specific Python concepts.

4.2 Core Gameplay Loop and Learning Mechanics

The prototype implements a structured gameplay loop designed to maintain continuous engagement. Students navigate the overworld map using keyboard controls, discovering Pokemon through environmental interactions such as tall grass, crystal boulders, and beached kelp. This exploration provides mental breaks between coding challenges while maintaining the game narrative.

When a Pokemon encounter occurs, the interface transitions to battle mode. Instead of traditional turn-based combat, students receive a region-specific programming task displayed in the code editor with clear task descriptions, starter code templates, expected output for self-assessment, and live syntax highlighting for immediate visual feedback.

The four regions organize tasks by Python concept: Forest focuses on for loop and iterations, Beach covers dictionary operations, Volcano introduces regular expressions, and Swamp presents advanced data structures including tuples and comprehensions. Students write Python code to solve the task, then click "Evaluate Code" to execute their solution. Upon successful validation, the Pokemon is captured and added to their inventory with calculated point rewards.

4.3 Design Principles Supporting Hyperfocus and Flow Experience

The prototype applies selected game design patterns and behavioral change techniques to support hyperfocus and sustained engagement. Hyperfocus is more likely when tasks feel rewarding, interesting, and engaging [12, 21] and is characterized by intense, narrow attention and improved task performance. The following design principles are intended to approximate these conditions and make such states more likely to occur.

Reinforcement and Gamification Systems.

The prototype employs multiple reinforcement mechanisms to support sustained engagement. Code execution provides immediate feedback with clear success or failure indicators, while Pokemon captures upon task completion create concrete pe-

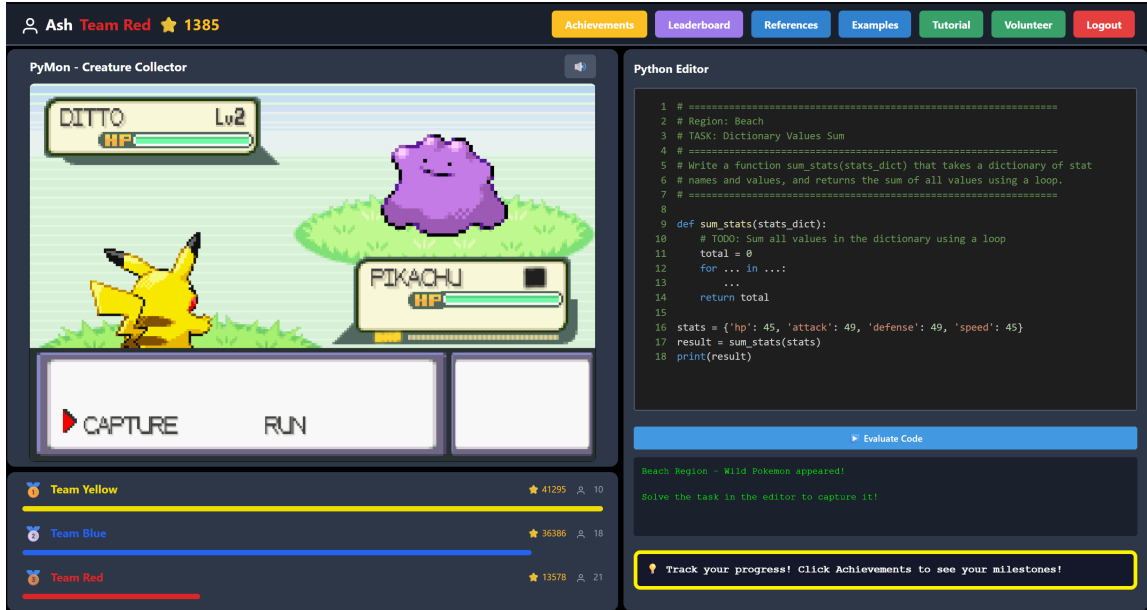


Figure 1: Snapshot of an encounter on the web app, the editor contains tasks that the user have to solve to capture the creature. (For better resolution, see Appendix B)

riodic rewards. The point system combines base points with time bonuses for faster solutions, rewarding both completion and efficiency. Real-time syntax highlighting, capture animations, and progress indicators provide continuous visual feedback throughout the coding process.

Collection mechanics create tangible progress as students build their Pokemon inventory. Regional mastery achievements unlock when students complete all six unique Pokemon within a region, while milestone tracking rewards captures (5, 10, 15, 20), login streaks (1-5 days), and playtime thresholds (60, 90, 120 minutes), contributing to 18 total unlockable achievements (see Appendix B, Figure 5 for the achievement interface).

Social Competition. PyMon incorporates two different leaderboards displaying individual trainer rankings and team-based aggregate scores. This design widens the possibility for students to find motivational engagement through either personal achievement or team contribution.

Temporal Engagement. Temporal mechanics support sustained engagement through daily login and session tracking that builds streaks, rewarding consistent practice patterns. Time tracking for task completion incentivizes optimization and multiple attempts. The timer is implemented as a hidden upward counter, maintaining performance tracking and enabling time-based bonuses without imposing visible time pressure that could disrupt focus states or create anxiety during problem-solving. This design decision was validated during pretesting, where participants expressed concern

that visible countdown timers would create unwanted pressure inconsistent with traditional programming learning environments.

Interface Design. The four-region structure categorizes tasks by Python concept, supporting focused practice within specific domains. Starter code templates reduce initial cognitive load. The single-screen integration with game and code editor side-by-side eliminates context switching that could disrupt focus states. Python syntax help and code examples remain accessible via modal on the top without requiring navigation away from the primary interface, (see Appendix B, Figure 6 for Examples modal).

4.4 Pretesting and Refinement

One week prior to the formal evaluation, the prototype underwent pretesting with five participants to identify usability issues and refine design elements. All five participants were computer science students who had completed bachelor's degrees. This pretesting phase revealed several critical insights that informed final design iterations, though it is important to note that the technical proficiency of these participants significantly exceeded that of the intended target audience of Python beginners.

Navigation. Spatial confusion emerged as a concern for at least two participants, who reported difficulty understanding the full scope of the game world. These participants described becoming "*stuck*" in the starting Forest region without awareness of other available areas. One participant noted:

"I sat a long time stuck in the Loop area, as I only thought that was the only thing that existed. I have no idea how big your world is."

This feedback directly informed the implementation of a Tutorial modal that provides game tutorials and displays a static map showing all four regions, helping players understand the full scope of the game world from the start.

Error Message Clarity. Error message clarity was identified as a usability issue by two participants. The terminal output display presented Python runtime errors alongside system-level debugging information from the code execution environment. At least two participants found this confusing, with one stating:

"Terminal output in the bottom right corner can seem very confusing with the error codes... there were many more errors that I didn't understand. I only need what is relevant to me, not the rest of your program."

Another participant also noted this concern: *"be aware that it also gives errors that don't really make sense."* In response to this feedback, an error filtering function was implemented in the code execution system to extract only task-relevant information. This aimed to present clearer feedback to novice programmers.

Task Variety and Randomization. Task repetition was noted by one participant, who reported encountering the same task three times within a short session: *"sitting and playing it right now... have gotten 3x count-vowels tasks, 2x in a row, another one, and then the same again."* This suggested potential issues with task pool size or randomization logic. In response to this feedback, the task pool was expanded from 24 to 34 unique tasks with randomization that excludes recently completed tasks, with the aim of providing better variety during longer gameplay sessions.

User Interface and Task Presentation. Interface clarity varied among participants. One participant noted difficulty locating task descriptions: *"don't know if I'm blind, but it took me a second to catch that there was a task description at the bottom of the screen."* In response, task descriptions were moved from the terminal output area to the code editor itself, where they appear as commented instructions at the top of each task, ensuring they remain visible even after code execution.

Additionally, one participant reported cursor synchronization issues in the code editor: *"the only thing that messed with me is that the cursor in the editor window is very out of sync. For example,*

here it should be right next to 'a' and not several spaces to the right." This was addressed by implementing synchronization between the text input area and the syntax highlighting overlay.

Despite their technical proficiency, two participants found some task descriptions unclear, with one commenting that *"some of the tasks are described a bit weirdly."* In response, task descriptions were revised to be more detailed, and scaffolding was provided through starter code templates that provide the structure for the solutions.

Validation of Design Decisions. The pretesting also provided support for several design elements. The leaderboard system appeared particularly motivating for two participants, with one reporting:

"Top Trainers created a lot of competition in me. I kept going because I wanted to rank higher."

Another participant suggested displaying player rank position and distance to the next rank to enhance competitive motivation. This feedback informed the implementation of an overall ranking display that shows each player's current position in the leaderboard, ensuring players always know their standing relative to others.

These pretesting insights shaped the final prototype implementation, particularly regarding navigation aids, better task description, and error message handling.

4.5 Technical Implementation

The prototype utilizes a React and TypeScript frontend with Pyodide for the browser Python execution, connected to a Godot-based game engine for the 2D gameplay. User data, task history, and achievements are persisted in a Supabase PostgreSQL database with real-time synchronization. This architecture enables immediate code execution feedback, supporting the uninterrupted attention characteristic of hyper-focus states.

The system implements 34 unique programming tasks distributed across the four regions: 10 tasks in Forest (loops and nested loops), 11 tasks in Beach (dictionary operations), 6 tasks in Volcano (regular expressions), and 7 tasks in Swamp (tuples and advanced data structures). This task pool size emerged from pretesting feedback indicating that participants encountered repetitive tasks with smaller sets. Each task is designed for beginner-level difficulty with estimated 2-5 minute solve times. Task selection uses database-level randomization with exclusion of recently completed tasks to maintain variety and prevent the immediate repetition observed during pretesting,

supporting the need for tasks to remain interesting, which is critical for triggering hyperfocus or flow states [10].

5 Results

This section presents the findings from the pilot study examining the potential of serious-gaming approaches to support hyperfocus and flow experiences in programming education. These analysis results include: 1) practical barriers and cognitive barriers are linked to task initiation difficulties, 2) the ways in which different types of breaks influence the continuity of engagement, and 3) game design elements that are indicative of supporting the continuity of engagement, while being highly individualized.

5.1 Lower Start-Up Friction Facilitates Entry into Deep Focus Experience

Participants mentioned during the interviews that the PyMon application eased task initiation by reducing both practical barriers (the physical steps required to begin) and cognitive barriers (the mental effort needed to start working) to begin programming learning exercises.

Current teaching practices in programming require multiple preparatory steps: opening the learning management system (Absalon), launching VS Code, locating course slides, and configuring the coding environment, a process that participants estimated took approximately 10 minutes before actual coding could begin. As Participant M, a 3rd year Mathematics student, explained:

"It's easier to load up a game than having to find all the [...] If I have to do exercise class, it always takes 10 minutes to load up everything and start VS code. Here you just open a game and then you can start coding. Because the gameplay here is of course actually coding. It leaves out all the boring stuff where you have to think about."

These preparatory steps, such as the coding editor setup, finding assignment descriptions, and estimation of time usage for solving programming tasks, require extra mental effort and delay the start of actual programming learning exercises. This means that the original intention and engagement in learning programming are hindered by practical, tedious tasks. The participants experienced that the game-based approach structured within 'one environment', which included both the tasks, the coding editor, and all relevant materials, reduced the effort required to initiate the task of practicing programming. The ready-to-use coding

interface with pre-configured templates removed preparatory tasks, which enabled students to start assignments more easily.

The preparatory steps as mentioned above delay the actual starting time of task initiation and create a gap between the **initiation of engagement** and the **enactment of engagement**. The initiation of engagement, in this context, refers to when motivation or readiness to engage emerges; in the students' case, this is the thought of starting to learn and practice programming. The enactment of engagement means the focused and sustained execution of actions that are related to the initiation of engagement. In the students' case, this is the actual programming learning and practice, solving programming tasks or reading lecture notes.

PyMon's 'one environment' game-based approach for programming task exercises removed the preparatory steps mentioned above from students' concerns and allowed students to focus solely on the action of programming. This characteristic of PyMon shows potential for lowering the risk of losing engagement, as participants might otherwise prefer to find something more interesting that requires less mental effort during the gap between engagement initiation and enactment, potentially leading to not starting to learn programming at all.

We observed that bridging the gap between engagement initiation and enactment through PyMon's 'one environment' game-based approach which might potentially point to positively influence the frequency of practicing programming exercise for students.

This observation is drawn on the participants' usage behavior, as observed through the temporal activity heatmap (Figure 2). The activity heatmap (Figure 2) provides a comprehensive view of engagement patterns across day-hour combinations from the testing period (N=41 registered players, 22 active players with non-zero points)

The heatmap shows a view of engagement patterns across day-hour combinations, showing that student usage sessions are widely distributed across the week, including both weekdays and weekends (Saturday: 2, Sunday: 2). The heatmap also shows that usage sessions are widely spread throughout each day. Aside from the mass concentration on Monday afternoon (when PyMon was introduced), the heatmap (Figure 2) shows engagement from early morning (6:00-8:00), prenoon (10:00-12:00), afternoon hours (14:00-17:00), and late evening periods (18:00-22:00). This observation suggests that students engaged with PyMon at varied times throughout the week. This

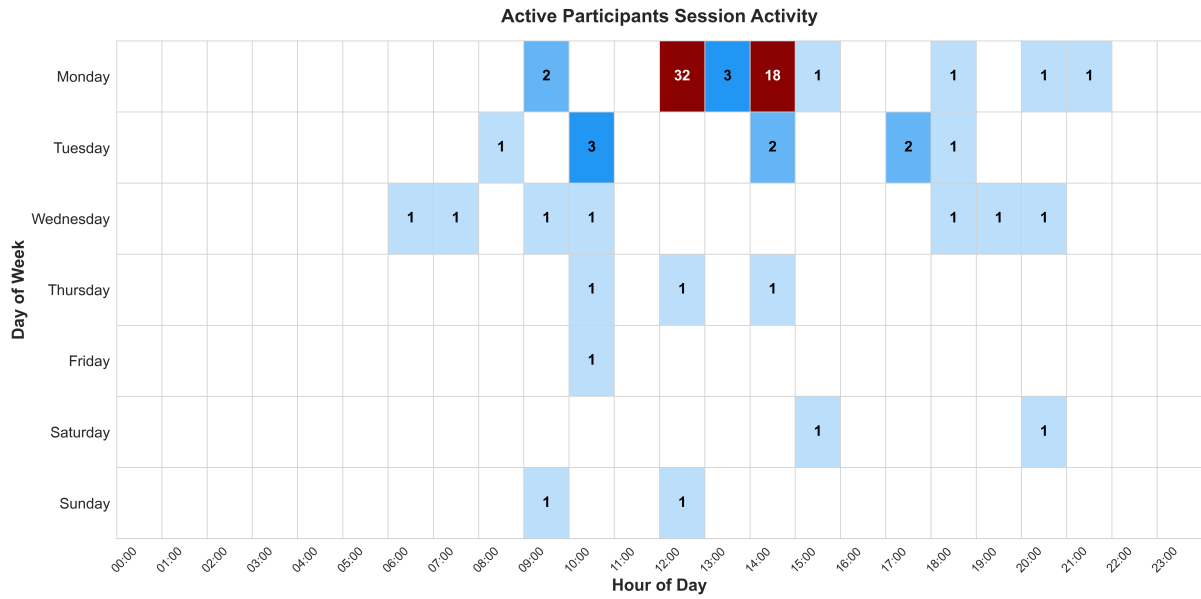


Figure 2: Activity heatmap showing session counts by day of week and hour of day. Darker colors indicate higher session counts. Monday midday (12:00-14:00) represents the most intensive usage period with 34 sessions at noon peak, accounting for 41% of all activity.

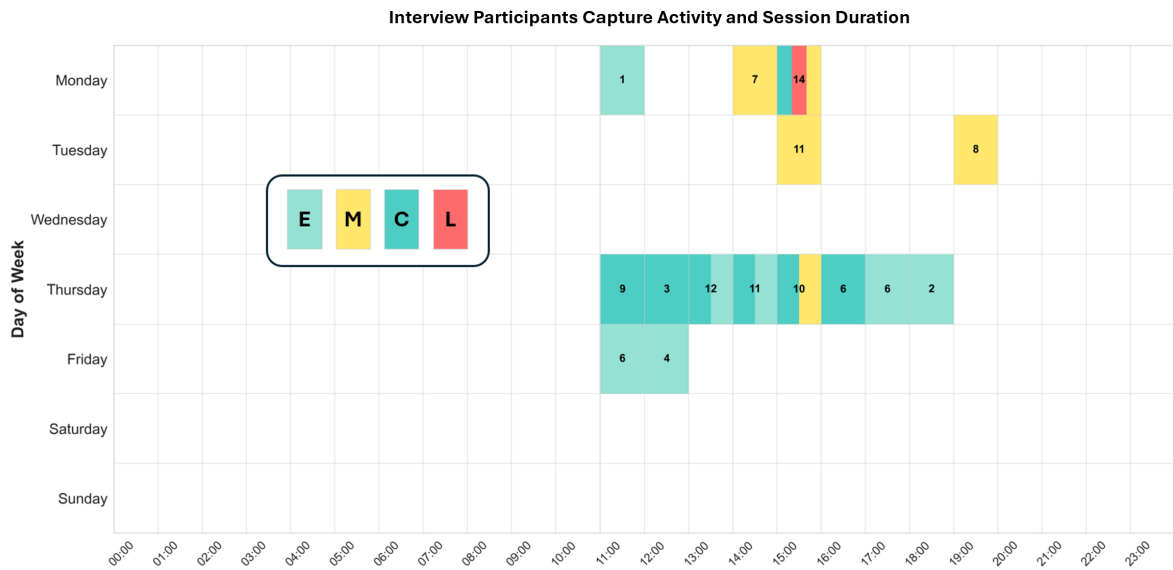


Figure 3: Interview participants' active session durations and Pokemon capture patterns based on actual capture timestamps. Numbers indicate Pokemon caught during each hour. Colors correspond to participants. Striped cells show combined captures from multiple participants.

temporal flexibility shows the potential for reducing barriers to task initiation, which may enable students to integrate short programming sessions into their busy weeks.

The observation of engagement patterns obtained from Figure 2 demonstrates the potential role of accessible, ready-to-use applications in increasing the likelihood of task initiation. By lowering the threshold for starting programming activity, which shortens the gap between engagement initiation and enactment, such design establishes a more favorable foundation for more frequent task

initiation.

Beyond practical barriers, psychological barriers also influence the participants' behavior when beginning programming learning sessions. One of the factors is the manageable time commitment, which is related to task size. PyMon's smaller, more granular tasks created lower psychological barriers to entry. Participant E, a 3rd year student from Molecular Biomedicine, noted:

"I could see that in PyMon, if you knew that would take a short amount of time, it was such a small task. It was seen as

less than starting an entire exercise or something like that, because those were sometimes a bit longer. This is really something I thought about when I did it."

The perception of manageable time commitment for the smaller programming tasks also has the potential to increase the likelihood of engagement to occur particularly during shorter availability windows. Figure 3 displays when they actually captured Pokemon based on capture timestamps, and participant M's capture pattern (Figure 3) exemplifies this opportunistic engagement. This distributed pattern across different days and times demonstrates how the reduced start-up friction for task initiation enabled students to engage with programming tasks when they had brief periods available, rather than requiring dedicated, extended study sessions.

Another factor through which PyMon assisted in lowering the psychological barriers for task initiation (i.e., starting to practice and learn programming) was the recontextualization of programming exercises from the participants' perspective. Through this recontextualization, participants no longer perceived engaging with PyMon for learning programming as equivalent to course-related programming activities, which they had previously described as demanding and associated with high mental load. Instead, programming tasks were reframed as game-like activities that provided participants with a more positive emotional experience and offered an alternative pathway toward the same engagement intention, but in a less aversive manner. As Participant M, a bachelor student from Mathematics, emphasized:

I think my experience was it was easier to stay focused on the task when using the app. [...] Turned it into a game which I found was nice. It was just like, I'm going to play a game now. And it's a lot easier to sit for more than half an hour playing a game probably because you don't necessarily feel like you were working in the same way. Yeah, so I thought it was easier to sit for longer times.

The participant's quote suggests that this shift in how programming exercises were conceptualized not only increased the willingness to engage with programming but also has the potential to enhance the actual frequency of initiating programming practice.

Our empirical data suggest that the combination of reduced setup steps, smaller and more manageable tasks, and the reconceptualization of pro-

gramming tasks has the potential to make it easier for students to start programming exercises. These conditions bridge the gap between engagement initiation and enactment by lowering both practical and psychological barriers, which supports fostering opportunities for sustained focus and engagement to emerge.

5.2 Context-Preserving Breaks Support Sustained Engagement

In the previous section, we discovered the potential of the serious gaming approach in facilitating lower starting friction that potentially supports the task initiation difficulties. In this section, we identified different types of breaks that influenced the continuity of engagement. Participants described experiencing two distinct types of breaks during programming sessions that differentially impacted their ability to maintain focus: *natural breaks* that preserved learning context, and *involuntary breaks* that disrupted it.

Natural Breaks The gameplay loop structure created intentional short breaks between programming tasks through Pokemon encounter sequences. These brief pauses provided mental refreshment while keeping students within the PyMon app rather than switching to external applications. As the participant M emphasized:

"I think that's [...] ease the mental load because you're getting a break, like a forced break in between every single task by just moving a bit around in the world which isn't mentally loading. [...] I'm still focused on the game. So whenever I get a new task in the game, I'm still immediately focused on that because that's my next objective in the game. So it's not because you are shifting from doing something else, it's more like you still know that you are practicing the game, but it's not because you missed the focus even though there was a break of it. "

However, objectively, breaks can be seen as interruptions to the intended practice of programming and as discontinuities in engagement. But from the students' view, since they had recontextualized the PyMon usage, these structured breaks were considered as part of the ongoing gameplay experience rather than a sequence of isolated programming tasks. As a result, these breaks did not function as disengagement points but instead offered brief mental rest periods that supported participants in subsequent tasks.

Compared to the worksheet-based workflow,

which encouraged participants to immediately progress from one task to the next with little cognitive decompression, as Participant M emphasized:

"Maybe it's probably that you're always, after you've done a task, if you're doing the worksheet, you just immediately move on to the next class or the next task while in the game you just move around a bit in the world. "

Participants' continuity of engagement was less likely to be disrupted in the game-based setting, as gameplay functioned as an ongoing experience aligned with the initial engagement intention (which was playing the game) when they initiated the task. In this context, short intended interruptions in the game-flow or transitions were experienced as part of the gameplay itself rather than as breaks from the primary objective. Unlike the worksheet-based problem-solving workflow, where the initial engagement intention is task completion, when a sub-task is done, this intention is cognitively perceived as being reached, which introduces natural stopping points where the gap between engagement intention and enactment appears. As a result, it is more likely to increase the likelihood of disengagement before the subsequent tasks are initiated.

Taken together, these findings indicate that the natural breaks embedded in the gameplay loop demonstrated potential in supporting the continuity of engagement while students practiced programming.

Rather than disrupting the learning process (in PyMon, this refers to the ongoing gameplay experience), these integrated brief pauses allow students to mentally reset while remaining in the same activity context. By preserving a consistent gameplay experience that aligns with the engagement initiation, it helps reduce opportunities for disengagement and supports sustained focus over time.

Involuntary Breaks Conversely, participants identified several factors that forced unplanned breaks and disrupted flow. These unplanned breaks can be caused by technical friction with the in-app code editor, particularly cursor alignment issues, which caused some participants to develop workarounds involving copying code to external editors (VS Code); unclear task instructions; or limited guidance within the PyMon app.

These mentioned factors introduce context switches that interrupt the continuity of engagement. These context switches again create the

gap between the engagement intention and the engagement enactment.

While both involuntary breaks and difficulties in task initiation break the continuity of engagement, a key distinction lies in when the disruption occurs. In this context, involuntary breaks force discontinuities after the bridging process between intention and enactment, rather than during the bridging process.

One scenario where involuntary breaks most frequently occurred was when participants became stuck on a particular task and were unable to make further progress in their programming work. In such situations, students were often forced to seek additional guidance. When the built-in support within PyMon was limited, students were pushed to seek external resources, thereby breaking the contained learning environment. Once this contained learning environment is no longer preserved, the discontinuity of engagement occurs, other activities could begin to draw students' attention and interest away from the task. As participant M described:

" Then suddenly you need to Google something up because you need to learn more to solve it. [...] When you load up Google and you start typing in something, the moment you switch a pane on your computer, your focus shifts and you can easily get distracted by something. Maybe you just had a thought, I have to search about this, I have to find gifts for Christmas anyway. And then you start looking for that suddenly. [...] Yeah, suddenly you get distracted. And you're like, oh, I should also have to check this or look this up or something. And then suddenly you've been doing something else for 15 minutes. and you're like, oh wait, I should go back to work. Is it hard to shift back to the focus for programming again [...] it's almost impossible to get started again, yeah. "

This quote illustrates that once the continuity of engagement is disrupted by context shifting, it becomes more difficult for students to re-engage with the original task. In particular, when involuntary breaks push students out of context against their intentions, they may become less aware of the disruption itself and more likely to be tempted to spend extended time on other stimuli that capture their attention. Such stuck scenarios not only increase the likelihood of distractions but also make it more difficult for students to sustain focus, as Participant E described:

" It depends if I was really stuck, if I have no idea how to even start, then my focus maybe be like 2 minutes and then I would (do other things)."

These observations suggest that discontinuities in engagement are closely related to participants' perception of current progress. The perception of current progress includes the clarity of next step that can be turned into doable actions. In contrast to the ambiguity about what to do next introduced by the involuntary breaks, participants' sense of progression was disrupted and key indicators of flow were undermined.

Accordingly, different types of breaks affected the continuity of engagement in distinct ways. A fundamental difference lies in learners' awareness of what will happen next and their sense of control. When this awareness is preserved, as in the structured breaks embedded within gameplay, these short interruptions do not disrupt focus but instead provide mental refreshment that supports continued engagement. In contrast, involuntary breaks often obscure the next step and provide the feeling of no progression, forcibly shifting attention toward alternatives, which enhances the likelihood of non-programming related activities and weakens commitment to the original intention.

5.3 Game Design Elements Dis- or Encourage Engagement but Show Individual Variation

In previous sections, we have introduced the gap between engagement intention and action and how different types of breaks influence it. In this section, we highlight and examine some of the game design elements that either supported or disrupted engagement according to indicators aligned with established measures of hyperfocus (AHQ) and flow (DFS-2). The indicators and their evidence from interviews are shown in Table 2. Interview data provided support for all hyperfocus dimensions (H1-H4) and most flow indicators (F1-F5).

Competitive Engagement For goal-oriented and competitively-motivated students, leaderboard mechanics produced intense, goal-directed engagement. Participant C, a 3rd year student from Molecular Biomedicine, described deliberately setting a goal to reach the top position:

"I just sat down on Thursday and then I was just grinding because I was like oh I want to get number one."

Similarly, Participant E reported: *"I became a bit competitive [...] I wanted to also get higher on the leaderboard."* For these participants, visible ranking created clear objectives and immedi-

ate feedback, which are both critical flow preconditions (F1, F2), that transformed programming practice into a competitive challenge. The leaderboard provided concrete markers of progress and comparative standing, fulfilling needs for achievement recognition. These behaviors indicate an alignment between engagement intention and its consistent enactment, which helped preserve the continuity of engagement. In this scenario, participants' engagement intention is to outperform other player within the game, while the corresponding engagement enactment involved continuously solving programming tasks to advance their ranking on the leaderboard. Likely, the next step for achieving the engagement intention for the students is clear, which is keep solving the programming tasks and gathering more points. This clearness of road-map for goal succession minimized the gaps that is created by the ambiguous next step like the scenario in stuck and support students' intrinsic motivation.

However, competitive elements did not universally motivate all participants; some participants did not consider ranking high on the leaderboard as their initial engagement intention. As Participant M indicated, the leaderboard *"didn't mean that much for my experience,"* though he acknowledged *"it's better than not having it."* Participant L, a beginner programmer, actively avoided the leaderboard during initial learning phases, suggesting that competitive pressure may create anxiety for beginners focused on skill development rather than performance comparison. Participants like M and L valued other design elements more, which suggests that how engagement can be influenced is related to individual preferences.

Achievement Systems and Progress Visibility

The achievement system provided additional motivation by recognizing when students reached specific milestones. Participants who valued having *"goals"* and *"quests to do"* appreciated the achievement structure, though several noted they would have engaged more if they could see what achievements were available to earn in advance. Participant C suggested: *"It would be cool if I could see what achievements I could try to get [...] then I would have more goals."* This feedback indicates that transparent achievement conditions could enhance the system's motivational impact by providing clearer objectives.

Pokemon collection (by owning different kinds of Pokemon in players' inventory in the game through solving programming exercises) served dual purposes: rewarding task completion while providing natural transition points between programming exercises. When paired with the in-

Indicator	Evidence from Interview
H1: Intense task locking	Participant C and Participant E maintained focus sessions of 1-2 hours; Participant M reported 30+ minute sessions; Participant C described "locked in" state when pursuing leaderboard goals
H2: Reduced external awareness	Participant C: <i>"I was pretty focused like when I had the goal in mind [...] then I like locked in"</i> ; Participant M noted being "completely focused" and "absorbed in something" to the point where "you don't notice time as much"
H3: Time transformation	Participant C: <i>"time passed really fast and suddenly two hours had passed"</i> ; Participant M: <i>"time always moves faster when you do that"</i> ; Participant E experienced Pokemon escape suggesting extended task duration beyond awareness
H4: Difficulty disengaging	Participant C described "hard to stop now" moments and grinding for multiple hours; Participant E continued playing after Pokemon escape to "get that one again"; Participant M found it "easier to sit for longer times" than traditional practice

(a) Hyperfocus Indicators

Indicator	Evidence from Interview
F1: Clear goals	The app organized tasks into distinct areas, each focusing on specific programming concepts (loops, dictionaries, regex). Participant C: <i>"I made my goal [...] I want to become number one on the leaderboard"</i> ; Participant E noted "it was nice to see that there are different areas where you have certain tasks"
F2: Performance feedback	Participant C: <i>"I especially liked the leaderboard [...] it gave me like a purpose"</i> ; Participant E: <i>"I like the achievements. I like that you could see progress [...] you could see them in an inventory so you could keep track"</i>
F3: Challenge-skill balance	Participant L valued "practicing in the same category to build mastery"; Participant M noted tasks were appropriate for beginner level but suggested "it would have been nice to get some harder exercises" for progression
F4: Sense of control	Participant L reported editor cursor alignment issues requiring workarounds (copying code to VS Code for editing, then pasting back); otherwise participants expressed autonomy in task selection and area navigation
F5: Intrinsic reward	Participant C: <i>"I want to get better at programming and especially like some of the more basic stuff"</i> ; Participant L: <i>"Learning new skills is the primary motivation"</i> with game mechanics providing supplementary engagement

(b) Flow Indicators

Table 2: Observed Hyperfocus and Flow Indicators from Interview Data

trinsic reward of learning and point accumulation, these elements created a "one more task" effect that reinforced persistence (F5). The combination of immediate feedback (Pokemon caught), visible progress (points earned), and achievement tracking supported sustained engagement by providing multiple reinforcement layers. Similarly to the leaderboard, the achievement system provides alignment of engagement intention and enactment, and provides a clear next step: to do one more task.

Editor Functionality & Insufficient Integrated Help Features Technical friction with the code editor integrated in PyMon emerged as a significant physical barrier to maintaining flow. Participant L, a 1st year Bioinformatics student, described cursor alignment issues that forced work-flow interruptions:

"I copy code to VS Code, work on it there, and paste it back [...] experienced

difficulty with the in-app editor's cursor alignment."

These disruptions introduced the very context-switching costs the PyMon app aimed to minimize and forced discontinuation of engagement enactment, suggesting that editor usability represents a critical prerequisite for sustained focus.

For novice programmers, syntax errors represented a particularly disruptive form of involuntary break. While this relates more to skill level than app design, participants noted that when the app's integrated help was insufficient, they resorted to external tools like ChatGPT, disrupting the focused programming context.

When PyMon successfully minimized these disruptive breaks through the integrated help features (Examples, References), some participants even described experiencing "hard to stop now" moments, which is a characteristic indicator of hyperfocus (H4).

The integrated help features further supported context preservation by enabling students to access learning resources without leaving the app environment as part of the gameplay experience. Participant C described how this design facilitated sustained engagement:

"If I was in a good flow and I like could find help on the app then I like stayed focused and then I did a lot of tasks in one day."

When students could resolve uncertainties within the PyMon app, they avoided the typical pattern of opening web browsers or course materials, transitions that participants consistently identified as triggers for distraction. In contrast, if they couldn't find help within PyMon and felt stuck (which was the most common experience for participants losing focus), as Participant E highlighted:

"When I felt stuck, mostly just trying again and again, then asking my friends for help or asking ChatGPT. [...] So just Googling help, looking at the PowerPoint and asking people if they could give any tips. "

As Participant M noted, switching to Google often led to browsing tangentially related content or checking social notifications, making it *"almost impossible to get started again"* when working at home.

These observations emphasize that discontinuities occurring during engagement enactment affect sustained focus. By integrating help and guidance within the application, PyMon provides conditions that minimize this type of engagement disruption.

Game Mechanic Calibration & Possibility for Saving code drafts While Pokemon escape mechanics (the programming task suddenly ends after a certain amount of time) aimed to create an appropriate challenge, participants indicated that excessive escape rates could disrupt focus during task transitions. Participant E's experience with the escape timer revealed the importance of calibrating difficulty to support rather than hinder engagement:

"I thought it was a bit annoying, actually, because I was very close to solving it [...] So it was just like, oh, I was so close to solving it, but now I can't remember exactly what it was."

However, E also noted that this mechanic motivated persistence: *"I think it made me stay longer because I was still annoyed that it happened. I*

was very like, I need to get that one again." This suggests that escape mechanics require careful balance to maintain engagement without causing frustration. This provides an interesting perspective, since Participant E was motivated by the leaderboard as the engagement intention. The escape of Pokemon does serve as an involuntary break that interrupts the continuity of engagement enactment, but this small disruption and setback result in higher interest in pursuing the engagement intention.

Participant M agreed that this escape mechanism might serve as a motivating factor but also acknowledged that it would be too annoying if this occurred too many times:

"If it's two minutes of work, I'm like, okay, whatever, that's funny that the Pokemon runs away, but if it [...] like 50 lines of code then I would have probably been frustrated. [...] If I find it funny when happening, it will probably motivate me to go again. I'll say, okay, I'm going to have to catch this guy now."

To address the problem of frustration over lost work, Participant M said:

"It can still escape, but I need to access the code I've already written in case I get the problem again at some point or something. [...] If it's a lot of code that just gets lost and isn't backed up somewhere, I will probably just, okay, I'm going to do something else now. So I think it really depends on whether the code is saved or not or not. If the code is saved, it will probably just motivate me, okay, I need to get this guy now. He can't escape this many times. So I think it would be a motivating factor as long as the code is backed up somewhere. "

The feature of saving created code in PyMon, combined with the escaping mechanism, might potentially increase engagement. The small penalty of being too slow in solving the programming task can be viewed as a next-time challenge if the written code is saved and can be retrieved for later use. Combining these two features would convert the disruption created by the forced quit task into a continued game experience without the demotivating feelings of starting over and wasting time.

Progressive Difficulty Scaling Participant M identified task repetition and lack of difficulty progression as limiting factors for sustained engagement:

"I couldn't find myself being in for more than half an hour because then I would start getting repetitions [...] I thought it would have been nice to get some harder exercises [...] some kind of progression."

This feedback indicates that maintaining the challenge-skill balance (F3) requires a structured difficulty progression to prevent both boredom from repetitive tasks and frustration from excessive difficulty.

Achievement Visibility Participant C suggested that transparent achievement criteria could enhance goal-setting opportunities:

"It would be cool if I could see what achievements I could try to get [...] then I would have more goals."

Similarly, Participant E requested, *"more achievements not just based on catching X amount of Pokemon or being on for a certain amount of time [...] like you have solved 5 loop assignments or 5 dictionary assignments."* These suggestions indicate that visible, varied achievement criteria could provide additional structure and motivation for sustained practice.

The design elements mentioned above reflect different paths for supporting engagement. Engagement patterns (Figure 3) revealed substantial individual differences in how participants interacted with the app. Competitively-oriented students (Participant C, Participant E) demonstrated intensive engagement sessions, with both recording 5-hour sessions on Thursday (Figure 3). Participant C concentrated all 46 captures within a single day (Figure 3). In contrast, Participant E distributed 24 captures across three different days (Figure 3). This is similar to Participant M, who also maintained distributed engagement across three days (Figure 3) but reported that task repetition and lack of difficulty progression limited continued engagement. This feedback indicates that challenge-skill balance (F3) requires careful calibration to sustain long-term interest across different user profiles. These individual differences highlight that game design approaches for initiating and preserving engagement are highly varied across different individuals. These findings indicate that there is not a universally effective design approach, but highlight the importance of accommodating and balancing diverse user motivations.

These findings suggest that game design elements do show indicators of encouraging engagement and supporting flow preconditions, but their impact is highly individualized. The implementation of the

application requires balancing the accommodation of diverse motivation profiles: providing competitive mechanics for achievement-oriented users while ensuring they do not create pressure or anxiety for learners focused on skill mastery.

5.4 Course Instructor Perspective

A semi-structured interview was conducted with the course instructor following the one-week testing period to gather insights on the potential integration of serious-gaming approaches into introductory programming courses. The instructor's perspective provides additional context for understanding potential institutional challenges and educational considerations.

The instructor reported limited engagement with the application during the testing period, using it twice for approximately 10 minutes each session. Despite this brief exposure, the instructor expressed positive reactions to specific design elements, noting the playfulness and connection to course content. The instructor particularly valued how the gameplay design appeared to avoid distracting students from programming tasks:

"You're not distracted by the gameplay [...] The gameplay leads to the programming problems, and then it's like a small reward to have the game universe to explore."

This observation appears consistent with student feedback about natural breaks potentially preserving learning context while providing mental refreshment between programming tasks.

However, the instructor identified student motivation as a primary challenge for implementing game-based learning tools in programming courses. The course serves a heterogeneous population with varying levels of intrinsic motivation:

"If they don't see a direct benefit right away from it, it's hard to convince them to use it [...] Students were not as keen to sort of take a risk. You know, they're very time optimized."

The instructor described multiple student categories within the course with different motivational profiles: bioinformatics students who may recognize programming's relevance to their future work; health informatics students who may not perceive programming as central to their studies; general science students seeking programming skills; and students from other departments seeking credit requirements. The instructor suggested this heterogeneity may create challenges for supplementary learning tools, particularly when students are uncertain whether games will help them

learn.

Beyond individual motivation, the instructor raised concerns about broader perceptions of learning through games:

"You're also facing that sort of challenge [...] the unfortunate mentality that [...] playful learning or game, isn't learning, right? And students, even though they don't want to learn traditional ways, they still have that mind-set."

The instructor suggested this tension between students' desire for engaging learning experiences and their internalized beliefs about what constitutes *legitimate* learning may present a barrier to adoption.

The instructor also raised questions about potential educational differences between game-based learning and traditional problem sets:

"In a problem set, you have to read the problem, you have to think about it, come up with a strategy [...] When you build the game, you chunk everything down to smaller bits."

The instructor suggested a possible trade-off: while chunking tasks into smaller units may reduce start-up friction (as some student participants reported), it might also reduce opportunities for students to develop problem-solving strategies across larger problem domains. The instructor noted this difference warrants further investigation and that the current prototype represents early exploration of this approach.

When asked about what data would be valuable for instructors, the course coordinator expressed interest in analytics about student learning processes, such as number of attempts, time spent, and correctness patterns. However, the instructor also cautioned against over-reliance on simple metrics:

"I'm not really sure that learning is like a performance sport [...] some days you have good days, and other days you have bad days [...] learning is complex."

Despite these challenges, the instructor expressed openness to continued exploration of game-based approaches, suggesting possibilities such as larger-scale testing or developing focused workshops on specific topics where game-based approaches might be appropriate.

6 Discussion

Our findings identified the gap between engagement initiation and enactment as the key factor influencing student engagement during programming practice. We identified three main mechanisms through which game-based design elements may reduce this gap and thereby enhance sustained engagement and focus: **(1)** facilitating task initiation by reducing practical and psychological barriers, **(2)** minimizing task shifting through context-preserving design, and **(3)** supporting diverse motivation profiles through customizable game mechanics. We discuss each mechanism in relation to existing theory and consider their implications for programming education.

6.1 Facilitating Task Initiation Through Reduced Barriers

6.1.1 Practical and Cognitive Start-Up Friction

Our findings suggest that the game-based approach may reduce barriers to initiating programming practice. Participants reported that the simplified setup process (opening a single web application rather than coordinating multiple tools) and smaller, discrete tasks lowered both practical and psychological barriers to initiating programming work. This finding appears particularly relevant for understanding the challenges faced by neurodivergent learners. Research indicates that individuals with ADHD often experience difficulty with task initiation, particularly when tasks require multiple preparatory steps or are perceived as time-intensive [14]. The executive function challenges associated with ADHD can make the transition from intention to action feel effortful, meaning that reducing setup requirements may provide some benefits for these learners.

The distributed engagement patterns observed in our data (Participant M's opportunistic practice across multiple days) suggest that when start-up friction is reduced, students may engage in programming practice during shorter availability windows rather than requiring dedicated study blocks. This pattern aligns with flow theory's emphasis on clear, immediately accessible goals (F1) as preconditions for deep engagement [2]. However, our sample was too small to determine whether this pattern was more pronounced among neurodivergent participants, and the one-week testing period limits conclusions about whether such patterns would sustain over time.

6.1.2 Programming as Play

An unexpected finding emerged regarding how participants conceptualized the activity itself. Participant M explicitly described the experience as *"playing a game"* rather than *"working,"* suggesting a cognitive re-framing of programming practice. This re-contextualization may reduce psychological barriers by shifting learners' emotional associations with the activity. Research on serious games suggests that playful framing can increase intrinsic motivation and reduce anxiety associated with academic performance [18].

This re-framing may also introduce the perception challenges discussed by our course instructor: if students view game-based learning as *"not legitimate learning,"* they may be less likely to engage with it despite its potential benefits. The instructor noted that students *"even though they don't want to learn traditional ways, they still have that mindset"* about what constitutes serious academic work. This suggests a societal perception barrier that extends beyond individual learner preferences. The effectiveness of game-based approaches may therefore depend not only on their educational benefits but also on how they are framed and integrated within institutional contexts.

6.2 Minimizing Task Shifting Through Context-Preserving Design

Our findings suggest a distinction between breaks that preserve learning context and those that disrupt it. Participants reported that Pokemon encounter sequences provided mental refreshment while keeping them within the learning environment, whereas technical issues or unclear instructions forced them to seek external resources, disrupting their concentration. This finding appears particularly relevant for neurodivergent learners. Research indicates that students with ADHD are particularly vulnerable to distraction when context-switching occurs [5], as they experience greater difficulty resuming interrupted tasks and are more prone to task-switching that disrupts sustained attention [14]. Participant M's description of finding it *"almost impossible to get started again"* after switching to Google illustrates this vulnerability.

Research on supporting executive function challenges recommends reducing extraneous cognitive load, minimizing transitions between contexts, and providing structured breaks that do not require navigating away from the primary task [5]. Our prototype's approach was designed with these principles in mind. The integrated help features aimed to support context preservation; however, participants noted gaps in the app, such as lacking

examples, that sometimes forced external searches anyway. This suggests that the completeness of in-app support systems may be important for maintaining flow, particularly for learners who are more susceptible to distraction. The challenge lies in providing sufficient scaffolding without creating mental overload from excessive information. Future iterations might explore adaptive help systems that adjust based on individual learner profiles and current task difficulty to address both completeness and cognitive load concerns.

However, our findings also revealed that involuntary breaks (editor issues, unclear instructions) disrupted flow regardless of the intended design. The editor cursor alignment issue reported by Participant L created the kind of disruptive context-switching that the PyMon app aimed to prevent. This suggests that technical polish is necessary for context preservation.

6.3 Supporting Diverse Motivation Profiles

6.3.1 Individual Variation in Game Element Effectiveness

Our findings revealed considerable individual variation in how game design elements influenced motivation and engagement. Competitive features (leaderboards) produced heightened engagement for some participants (C, E), while others (M, L) reported limited or even negative effects. This variation appears consistent with existing research showing that game-based motivational elements interact with individual differences in competitiveness, skill level, and learning goals [9].

The challenge-skill balance (F3) emerged as particularly important for sustained engagement. Participant M's feedback regarding task repetition becoming demotivating after 30 minutes suggests that maintaining appropriate challenge levels requires dynamic difficulty adjustment or progressive content structures. This finding relates to flow theory's emphasis on calibrating challenge to skill level [2], though our prototype's static difficulty curve limited opportunities to test this relationship systematically.

For neurodivergent learners specifically, individual variation may be even more pronounced. Research emphasizes that neurodivergent individuals, while sharing some common patterns, exhibit highly individualized responses to interventions and support structures [18]. This suggests that one-size-fits-all game mechanics may be insufficient, instead, customizable or adaptive game elements that accommodate different motivation profiles and skill progressions may be necessary.

6.3.2 Balancing Competitive and Mastery

The contrast between competitive engagement (C, E) and mastery-focused learning (L) highlights a tension in game design for education. Competition-oriented features can encourage sustained practice for some learners while creating anxiety or distraction for others focused on skill development. Participant L's active avoidance of the leaderboard during learning suggests that competitive pressure may interfere with learning for beginners who need to focus on understanding rather than performance comparison.

This finding suggests that effective game-based learning tools may need to support multiple pathways to engagement: competitive mechanics for competition-oriented learners and mastery-based progression for skill-focused learners. However, implementing this flexibility without creating interface complexity presents a significant design challenge. The tension between providing options and maintaining coherent design deserves further investigation.

6.4 Educational Trade-offs: Task Chunking vs. Problem-Solving Development

The course instructor raised important questions about potential educational differences between game-based learning and traditional problem sets. The instructor suggested that while games *"chunk everything down to smaller bits,"* traditional problem sets require students to *"read the problem, think about it, come up with a strategy"* in an integrated way. This observation points to a potential tension between design elements that reduce start-up friction (smaller, discrete tasks) and learning objectives that emphasize holistic problem-solving skills.

From a cognitive load perspective, chunking complex tasks into smaller units can make them more accessible by reducing working memory demands and providing more frequent success experiences [18]. This may be particularly beneficial for learners with ADHD, who often experience working memory limitations [1]. However, if chunking is too granular, students may complete individual tasks without developing the integrative thinking skills needed to tackle larger, open-ended problems.

This tension may not be irreconcilable. It is possible that game-based approaches could serve complementary roles in programming education rather than replacing traditional problem sets entirely. Games might be most effective for building foundational skills, practicing basic concepts, and maintaining engagement during routine practice, while traditional problem sets could focus on de-

veloping strategic thinking, integration skills, and complex debugging.

6.5 Hyperfocus and Flow: Evidence and Limitations

Our interview data provided small evidence consistent with both hyperfocus and flow indicators. Participants reported time transformation, reduced external awareness, difficulty disengaging, and experiences of being *"locked in"* during programming sessions. However, examining these findings through the theoretical distinctions between flow and hyperfocus suggests that the serious gaming context may be particularly well-suited to facilitating flow states rather than hyperfocus.

As discussed in the theory section, flow is characterized as an optimal and controlled engagement state supported by clear goals, immediate feedback, and balanced challenge relative to skill level [2]. In contrast, hyperfocus involves stronger attentional locking and greater difficulty disengaging, which can be productive in the moment but may be less flexible or harder to regulate [12, 2]. Our prototype's design elements, including explicit task goals (F1), immediate feedback through Pokemon captures and points (F2), progressive difficulty through different areas (F3), and integrated help features (F4), align closely with the preconditions for flow. These design features create the structured, goal-oriented environment that flow theory emphasizes, making flow a more manageable outcome in serious gaming contexts compared to hyperfocus, which tends to emerge more spontaneously in personally rewarding activities [10].

The evidence from our participants supports this interpretation. While participants reported intense concentration and time distortion, these experiences occurred within a context that provided clear objectives, regular feedback, and manageable challenges. The fact that competitive participants (C, E) described setting specific goals and tracking progress toward them through the leaderboard suggests flow-like engagement rather than the less controlled attentional locking characteristic of hyperfocus. Similarly, the short breaks through Pokemon encounters provided mental refreshment while maintaining task continuity, which aligns with flow's emphasis on smooth, controlled engagement rather than the difficulty disengaging associated with hyperfocus.

However, several limitations constrain our ability to draw strong conclusions about which state participants experienced or whether one is more desirable than the other in educational contexts.

First, we lack comparison data from the same participants using traditional programming practice methods, making it difficult to determine whether reported focus experiences were specific to the game-based approach or would occur regardless of the interface. Second, the one-week testing period and novelty of the tool may have influenced engagement in ways that would not sustain over longer periods. Third, throughout our interviews, we did not explicitly distinguish between flow and hyperfocus experiences, meaning participants may have experienced elements of both states or moved between them during different sessions.

Additionally, while some participants described intense engagement sessions, we cannot determine whether these represented optimal learning states. The instructor's caution that "*learning is not like a performance sport*" reminds us that sustained time-on-task does not necessarily equate to effective learning. Participants might have experienced flow-like engagement while practicing tasks they had already mastered (as Participant M reported) without developing new skills.

6.6 Institutional and Adoption Challenges

Beyond individual learner experiences, our findings highlight institutional factors that may influence whether game-based approaches can be effectively integrated into programming education. The course instructor identified several barriers: different student populations with varying motivation, time-optimization pressures that make students risk-averse about trying new tools, and societal perceptions that devalue playful learning approaches.

These findings suggest that even educationally effective tools may face adoption challenges if they require students to invest time without guaranteed benefits or if they conflict with students' internalized beliefs about legitimate learning. The instructor's observation that students "*don't see a direct benefit right away*" points to an expectation for immediate, visible outcomes that may be unrealistic for learning tools. Addressing these barriers may require institutional support, such as explicit instructor endorsement, integration into course structure rather than optional supplements, and clear communication about learning objectives and benefits.

6.7 Implications for Design and Practice

Based on our findings, we offer several considerations for designing game-based programming education tools:

Reduce Setup Friction: Integrated, web-based

interfaces that eliminate multi-tool coordination may lower barriers to practice initiation, particularly for learners who struggle with task initiation or executive function challenges.

Prioritize Context Preservation: Design choices should minimize forced transitions out of the learning environment. This requires comprehensive in-app support systems and technical polish to avoid creating involuntary interruptions.

Support Multiple Motivation Pathways: Rather than assuming all learners respond similarly to competitive or achievement-oriented mechanics, consider providing customizable features or multiple progression pathways that accommodate different learning orientations.

Balance Chunking with Integration: While smaller tasks may reduce start-up friction and maintain engagement, consider how to scaffold toward larger, more integrative problem-solving. Graduated difficulty progressions that increase both task complexity and integration requirements over time may help bridge this gap.

Address Perception Barriers: Work with instructors to frame game-based tools as legitimate learning resources and integrate them meaningfully into course structures rather than positioning them as optional supplements.

However, these considerations emerge from a limited pilot study and should be tested more rigorously before being adopted as design principles.

6.8 Universal Design and Neurodiversity

While our study was motivated by challenges faced by neurodivergent learners, several findings (reduced setup friction, context preservation, multiple engagement pathways) may benefit all learners. This aligns with universal design principles: accommodations designed for learners with specific needs often improve experiences for everyone [5]. Rather than creating separate tools for neurodivergent learners, designing with neurodiversity in mind from the outset may produce more flexible, accessible learning environments for all students.

6.9 Future Directions

This pilot study has several limitations that suggest directions for future research. The small participant sample (N=4 interviews, N=41 total users) and one-week testing period limit generalizability and prevent conclusions about sustained engagement or learning outcomes. The lack of controlled comparison conditions means we cannot definitively attribute observed effects to the

game-based approach rather than novelty or other factors.

Future research could address these limitations through larger-scale studies with control groups, tracking engagement and learning outcomes, explicit assessment of skill transfer to traditional programming contexts, and investigation of how individual differences (including prior programming experience and learning orientations) interact with responses to game-based approaches. Additionally, systematic variation of specific design elements, such as comparing chunked vs. integrated tasks or competitive vs. mastery-oriented mechanics, could help identify which features contribute most to engagement and learning.

7 Conclusion

This study explored the potential of serious-gaming approaches to support STEM students in experiencing hyperfocus or flow while working on programming assignments. Through a one-week pilot study with our web-based prototype PyMon, we identified three mechanisms through which game-based design may influence engagement, which serve as implications for designing future programming education tools: **(1)** facilitating task initiation by reducing setup friction through integrated environments; **(2)** minimizing task shifting by preserving learning context with comprehensive in-app support; and **(3)** supporting diverse motivation profiles through customizable designs. Participants reported focus experiences consistent with flow and hyperfocus indicators, although the limited testing period and lack of control group prevent firm conclusions about causal effects.

Our findings suggest that game-based approaches may serve complementary rather than replacement roles in programming education. While reduced barriers and integrated environments appear beneficial for initiating and sustaining practice, tensions exist between task chunking that lowers friction and educational goals that focus on problem-solving. The considerable individual variation observed; where competitive features engaged some learners while creating anxiety for others, this indicate that effective tools must accommodate multiple engagement types rather than assuming universal responses.

Beyond design considerations, institutional factors emerged as potentially significant adoption barriers, including societal perceptions that devalue playful learning and time-optimization pressures that make students risk-averse about trying new approaches. These findings highlight that educational effectiveness alone may be insufficient with-

out attention to institutional integration, and perceived legitimacy.

This study contributes to understanding how universal design principles informed by neurodiversity research may benefit all learners. The high representation of neurodivergent students in STEM presents an opportunity to design more inclusive programming education that recognizes diverse cognitive profiles as central considerations rather than special cases.

References

- [1] E. Alabdulkareem and M. Jamjoom. Computer-assisted learning for improving adhd individuals,Äô executive functions through gamified interventions: A review. *Entertainment Computing*, 33:100341, 2020. ISSN 1875-9521. doi: <https://doi.org/10.1016/j.entcom.2020.100341>. URL <https://www.sciencedirect.com/science/article/pii/S1875952119300953>.
- [2] B. K. Ashinoff and A. Abu-Akel. Hyperfocus: The forgotten frontier of attention. *Psychological research*, 85(1):1–19, 2021.
- [3] M. Beltrán. Defining, classifying and identifying addictive patterns in digital products. *IEEE Transactions on Technology and Society*, 2025.
- [4] P. Bjørn, K. F. Larsen, and M. Msfelt. Diversity, equity and inclusion (dei) in computer science: Bridging gender and neurodiversity. *NeuroGen Computing*, 2025.
- [5] V. Borsotti, A. Begel, and P. Bjørn. Neurodiversity and the accessible university: Exploring organizational barriers, access labor and opportunities for change. *Proceedings of the ACM on Human-Computer Interaction*, 8 (CSCW1), 2024. doi: 10.1145/3641011. URL <https://doi.org/10.1145/3641011>.
- [6] V. Braun and V. Clarke. Using thematic analysis in psychology. *Qualitative Research in Psychology*, 3(2):77–101, 2006. doi: 10.1191/1478088706qp0630a. URL <https://www.tandfonline.com/doi/abs/10.1191/1478088706qp0630a>.
- [7] D. Cohen and B. Crabtree. Qualitative research guidelines project, July 2006. URL <http://www.qualres.org/index.html>. Accessed: 27 December 2025.
- [8] Department of Computer Science, University of Copenhagen. Ansøgnings- og optagelsestest, 2025. URL <https://di.ku>.

- dk/ominstituttet/ansoegningstal/. Accessed: 19 December 2025.
- [9] R. Garris, R. Ahlers, and J. E. Driskell. Games, motivation, and learning: A research and practice model. In *Simulation in aviation training*, pages 475–501. Routledge, 2017.
 - [10] Y. Groen, U. Priegnitz, A. B. M. Fuermaier, L. Tucha, O. Tucha, S. Aschenbrenner, M. Weisbrod, and M. Garcia Pimenta. Testing the relation between adhd and hyperfocus experiences. *Research in Developmental Disabilities*, 107:103789, 2020. doi: 10.1016/j.ridd.2020.103789.
 - [11] M. M. Grotewiel, M. E. Crenshaw, A. Dorsey, and E. Street. Experiences of hyperfocus and flow in college students with and without attention deficit hyperactivity disorder (adhd). *Current Psychology*, 42:13265–13275, 2023. doi: 10.1007/s12144-021-02539-0.
 - [12] K. E. Hupfeld, T. R. Abagis, and P. Shah. Living “in the zone”: Hyperfocus in adult adhd. *ADHD Attention Deficit and Hyperactivity Disorders*, 11(2):191–208, 2019. doi: 10.1007/s12402-018-0272-y.
 - [13] J. Koivisto and J. Hamari. The rise of motivational information systems: A review of gamification research. *International Journal of Information Management*, 45: 191–210, 2019. ISSN 0268-4012. doi: 10.1016/j.ijinfomgt.2018.10.013. URL <https://www.sciencedirect.com/science/article/pii/S0268401217305169>.
 - [14] K. J. Maw, G. Beattie, and E. J. Burns. Cognitive strengths in neurodevelopmental disorders, conditions and differences: A critical review. *Neuropsychologia*, 197:108850, 2024.
 - [15] K. J. Maw, G. Beattie, and E. J. Burns. Cognitive strengths in neurodevelopmental disorders, conditions and differences: A critical review. *Neuropsychologia*, 197:108850, 2024.
 - [16] R. M. Mettenbrink. Prevalence of autism spectrum characteristics in students taking undergraduate computing courses. Master’s thesis, University of Nebraska–Lincoln, 2024. URL <https://digitalcommons.unl.edu/computerscidiss/243>. Master’s thesis, DigitalCommons@University of Nebraska–Lincoln.
 - [17] H. Noble and R. Heale. Triangulation in research, with examples, 2019.
 - [18] L. C. Rodríguez Timaná, J. F. Castillo García, T. Bastos Filho, A. A. Ocampo González, N. R. Hincapié Monsalve, and N. J. Valencia Jimenez. Use of serious games in interventions of executive functions in neurodiverse children: systematic review. *JMIR Serious Games*, 12:e59053, 2024.
 - [19] J. A. Sedgwick, A. Merwood, and P. Asherson. The positive aspects of attention deficit hyperactivity disorder: a qualitative investigation of successful adults with adhd. *ADHD Attention Deficit and Hyperactivity Disorders*, 11(3):241–253, 2019.
 - [20] Y. Shin, J. Jung, J. Zumbach, and E. Yi. The effects of worked-out example and metacognitive scaffolding on problem-solving programming. *Journal of Educational Computing Research*, 61(6):1312–1331, 2023. doi: 10.1177/07356331231174454. URL <https://doi.org/10.1177/07356331231174454>.
 - [21] R. H. Sklar. *Hyperfocus in adult ADHD: An EEG study of the differences in cortical activity in resting and arousal states*. University of Johannesburg (South Africa), 2013.
 - [22] A. Tashakkori and J. W. Creswell. The new era of mixed methods, 2007.
 - [23] A. J. Van Rooij, M. V. Birk, S. Hof, S. Ouburg, and S. v. Hilten. Behavioral design in video games: A roadmap for ethical and responsible games that contribute to long-term consumer health and well-being. *Behavioral design in video games: a roadmap for ethical and responsible games that contribute to long-term consumer health and well-being*, 2021.
 - [24] X. Wei, J. W. Yu, P. Shattuck, M. McCracken, and J. Blackorby. Science, technology, engineering, and mathematics (stem) participation among college students with an autism spectrum disorder. *Journal of Autism and Developmental Disorders*, 43(7):1539–1546, 2013. doi: 10.1007/s10803-012-1700-z.
 - [25] J. P. Zagal, S. Björk, and C. Lewis. Dark patterns in the design of games. In *Proceedings of the International Conference on Foundations of Digital Games*, 2013.

8 Appendix

A Interview Guide

A.1 Research Question

What are the potentials of using serious-gaming approaches to support STEM students in experiencing hyperfocus or flow while working on programming tasks?

A.2 Interview Questions

1. **Tell me about the last programming session before you used the app (in class or at home). What did you work on, and how did your focus play out from start to finish?**
 - Typical session length, common break points
 - Baseline focus
2. **How did working with the app feel compared to your usual practice?**
 - What felt smoother, what felt more effortful, moments of friction
 - Overall app impact on focus
3. **Which parts of the app helped you stay with the task, and which pulled you out?**
 - Immediate feedback, goals/instructions, rewards/streaks, collection, leaderboard/teams, hints, timers, UI clarity
 - Design elements → focus/defocus
 - Why do you think that element had that effect?
4. **Was there a moment you felt highly absorbed during programming with the app? What exactly was happening on screen and in the task?**
 - What triggered it, how it ended, interruptions, environment
 - Absorption, action–awareness merging
 - At that moment, would it have been easy or hard to pause or switch tasks?
5. **While using the app, how did time feel—faster, slower, or normal?**
 - Estimate felt vs. actual duration
 - Time transformation / time loss
6. **Describe a moment your focus broke. What caused it?**
 - Unclear goals; confusing instructions; UI clutter; timer pressure; social features; external distractions
 - Breakers of flow/hyperfocus
 - What did you try/do next, and why?
7. **How did the task difficulty feel relative to your skills?**
 - Too easy/too hard moments, use of hints, switching tasks
 - Challenge–skill balance; sense of control
8. **What made you continue to the next task? Or decide to stop?**
 - Rewards/collection, leaderboard pressure, curiosity, narrative, feedback cadence, fatigue
 - Drivers of persistence vs. disengagement
 - Tell me the story of the moment you decided to stop

9. If you could change any feature to help you stay in deep focus while programming, what would it be?

A.3 Wrap-up Question

Is there anything about your experience of deep focus with the app that we did not ask about but should know?

A.4 Theoretical Framework Indicators

A.4.1 Hyperfocus (Adult Hyperfocus Questionnaire - AHQ)

- **H1.** My attention felt intensely locked on the task.
- **H2.** I was less aware of things around me.
- **H3.** Time slipped by / I lost track of time.
- **H4.** It would have been hard to disengage from the tasks (even if you needed to).

A.4.2 Flow (Dispositional Flow Scale-2 - DFS-2)

- **F1.** I knew clearly what I needed to do next.
- **F2.** I could tell how well I was doing while working.
- **F3.** The difficulty felt well matched to my skills.
- **F4.** I felt in control of my actions.
- **F5.** The experience itself was rewarding. (optional)

B Prototype Interface

The screenshot displays the PyMon web application interface, which is divided into several sections:

- Top Navigation Bar:** Contains links for [Logout](#) (red), [Volunteer](#) (green), [Tutorial](#) (green), [Examples](#) (blue), [References](#) (blue), [Leaderboard](#) (purple), [Achievements](#) (yellow), and a user profile section for **Ash Team Red** with a star icon and the number **1385**.
- Python Editor:** A central area with a dark background for writing code. It includes a **Python Editor** title, a **Run** button, and an **Evaluate Code** button. The code in the editor is as follows:

```
1 # =====
2 # Region: Beach
3 # TASK: Dictionary Values Sum
4 # =====
5 # Write a function sum_stats(stats_dict) that takes a dictionary of stat
6 # names and values, and returns the sum of all values using a loop.
7 # =====
8
9 def sum_stats(stats_dict):
10     # TODO: Sum all values in the dictionary using a loop
11     total = 0
12     for ... in ...:
13         ...
14     return total
15
16 stats = {'hp': 45, 'attack': 49, 'defense': 49, 'speed': 45}
17 result = sum_stats(stats)
18 print(result)
```
- Encounter Simulation:** A central area showing a Pokemon encounter. It features a **PyMon - Creature Collector** title, a **PyMon** logo, and a **PyMon** status bar. The encounter shows a **PIKACHU** (yellow) and a **PIKACHU** (purple) with their respective HP bars. Below the encounter, there are **CAPTURE** and **RUN** buttons.
- Team Status:** A section at the bottom showing the status of three teams: **Team Yellow** (41295 stars, 10 lives), **Team Blue** (36306 stars, 18 lives), and **Team Red** (13578 stars, 21 lives). Each team has a corresponding colored bar and a **PyMon** icon.
- Footer:** A yellow bar at the bottom with the text: **Track your progress! Click Achievements to see your milestones!**

Figure 4: Snapshot of an encounter on the web app, the editor contains tasks that the user have to solve to capture the creature.

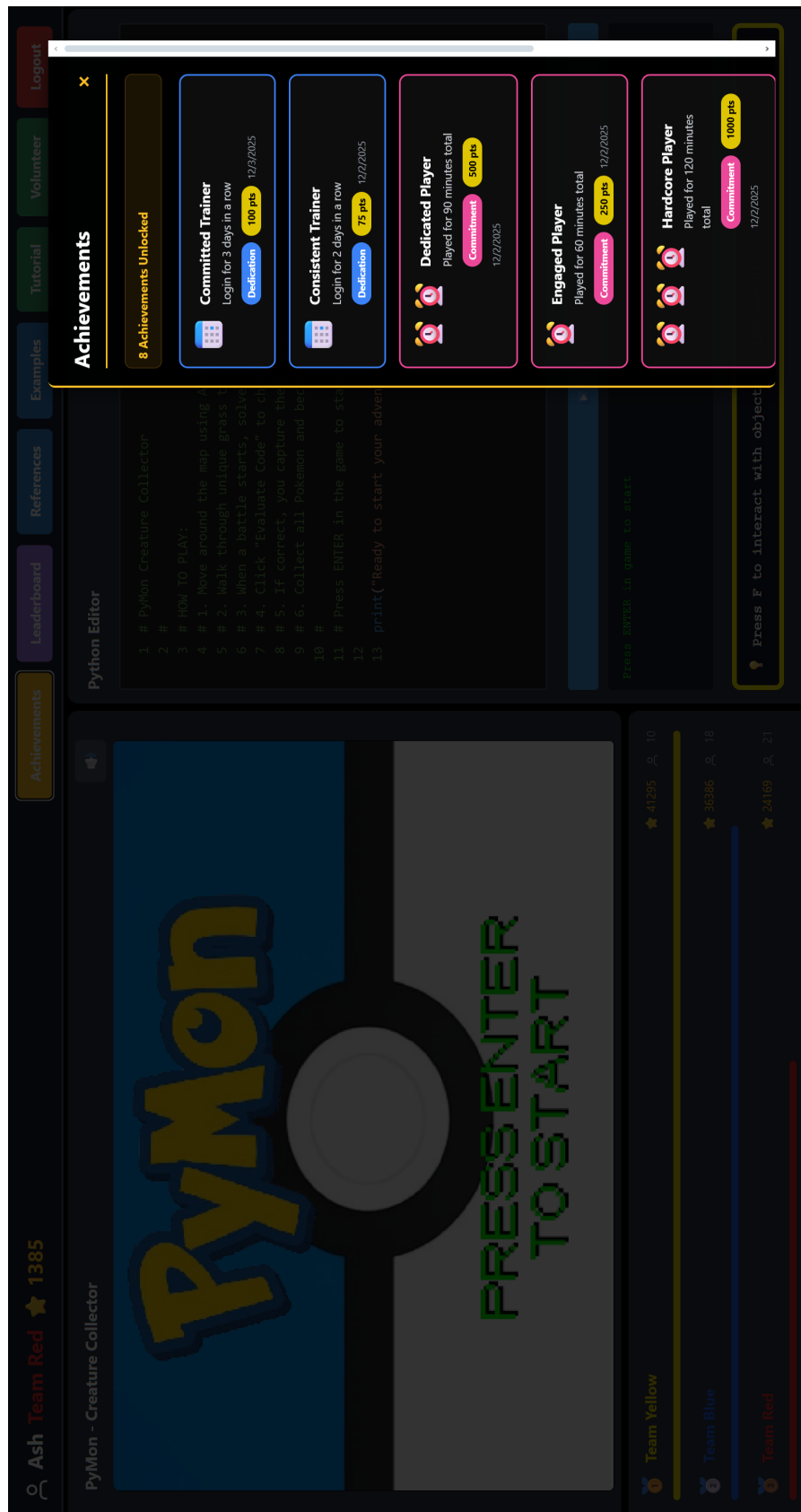
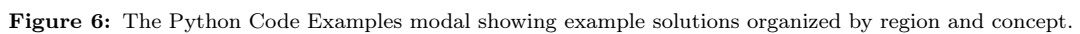


Figure 5: The Achievements interface showing unlocked achievements including login streaks and playtime milestones. Each achievement displays its unlock criteria, point value, and completion date.



UCPH's AI declaration

Declaration of using generative AI tools
☒ I/we have used generative AI as an aid/tool (please tick) ☐ I/we have <u>NOT</u> used generative AI as an aid/tool (please tick) <i>If generative AI is permitted in the exam, but you haven't used it in your exam paper, you just need to tick the box stating that you have not used GAI. You don't have to fill in the rest.</i>
List which GAI tools you have used and include the link to the platform (if possible): GitHub Copilot (connected via my GitHub account linked to UCPH email): https://github.com/features/copilot Grammarly: https://www.grammarly.com/ Otter AI: https://otter.ai/ WhisperTranscriber (KU Digi Labs): https://digilabs.ku.dk/labs/observe-lab/#wt
Describe how generative AI has been used in the exam paper: We primarily used AI tools for grammar correction and coding inspiration. For grammar correction, we used GitHub Copilot in VS Code and Grammarly during the writing process to help identify and fix grammatical errors in our report. Copilot also suggested alternative sentence structures, which helped us improve the flow of our writing. As an example, we used these tools to check for correct tense usage, as we sometimes accidentally used present tense instead of past tense. We used these tools mainly during the writing phase to polish our text before supervisor meetings and before final submission. When reviewing suggestions from Copilot and Grammarly, we manually edited them to ensure they fit well with the overall flow and tone of our report. For coding inspiration, we used GitHub Copilot during the development phase of PyMon for coding assistance. For example, when writing the programming tasks for the game, we created task examples that would be useful for the users. For this, we used Copilot to suggest code snippets to be used for training examples. We then evaluated these code snippets, tested them to make sure they were appropriate for our users skill level.

For interview transcription, we used Otter AI and WhisperTranscriber from KU Digi Labs to transcribe our interview recordings. This allowed us to quickly convert audio interviews with students and instructors into text format for our qualitative analysis. We reviewed the automated transcripts, as automated transcription can contain errors, especially with technical terminology or non-English sentences.

Please note: Content generated by GAI that is used as a source in the paper requires correct use of quotation marks and source referencing. Read the guidelines from Copenhagen University Library at KUnet [here](#).